Woxsen University

School of Technology

B.Tech – Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Title of the Project

(Vehicle Service Management System)

Course Code: _____

Course Name: Web Technologies

Semester: III

Academic Year: 2026-27

Submitted by:

1. Joel – 3011

2. Jayanth – 3021

3. Sayan Bhowmick – 3035

4. Hansith – 3041

Section: _____

Under the Guidance of:

Faculty Name: _____

Designation: _____

Department of Computer Science and Engineering

October, 2026

CERTIFICATE

This is to certify that the Project titled "**Vehicle Service Management System**" is a bonafide work carried out by **Joel (Register No: 3011); Jayanth (Register No: 3021); Sayan Bhowmick (Register No: 3035); Hansith (Register No: 3041)**, B.Tech – Computer Science and Engineering, during the Academic Year **2026-27**, under my supervision and guidance.

Guide Signature

(Name of the Guide)

Designation

DECLARATION

We hereby declare that the Project Title "**Vehicle Service Management System**" submitted by us is an original work carried out under the guidance of the faculty mentioned above.

We confirm that this work has not been submitted in part or full for the award of any degree, diploma, or other academic qualification in this or any other institution.

All sources of information used in this case study have been properly acknowledged and cited as per university referencing standards.

We further declare that the plagiarism percentage is within the permissible limit prescribed by the institution.

Place: _____

Date: _____

Signature of the Candidates

Name	**Register Number**	**Signature**
Joel	3011	_____
Jayanth	3021	_____
Sayan Bhowmick	3035	_____
Hansith	3041	_____

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty guide for the guidance, feedback and encouragement given throughout this project, and for the reviews that shaped both the design and this report.

We thank the Department of Computer Science and Engineering, School of Technology, Woxsen University, for the opportunity to carry out this work as part of the Web Technologies course, and for the facilities provided.

We also thank our classmates for testing the application and pointing out problems we had missed.

ABSTRACT

Vehicle service centres still run largely on telephone calls and paper job cards. Customers cannot see what is being done to their vehicle, front-desk staff double-book mechanics, parts run out in the middle of a job, and the owner has no single view of revenue, workload or turnaround. This project designs and builds AutoCare, a web-based Vehicle Service Management System in which customers, receptionists, mechanics and the manager work from the same live record, each limited to what their role permits. The client is a React 19 and TypeScript single-page application styled with Tailwind CSS 4 and a token-based design system with light and dark themes; it is served by Vite, which proxies API calls so that the browser sees a single origin. The server is a FastAPI application exposing 33 JSON endpoints over MongoDB, configured as a replica set so that business rules run inside multi-document transactions. Three rules are enforced on the server: a mechanic cannot hold overlapping assignments, a part cannot be issued beyond stock on hand, and invoice totals are computed rather than accepted from the client. Authentication uses JSON Web Tokens; authorisation checks both the caller's role and, for customers, ownership of the record requested. The system was evaluated by measurement rather than assertion: 28 automated tests pass; median API latency is under 6 ms for every read endpoint; loading the dashboard's seven reports in parallel takes 9.2 ms against 19.2 ms sequentially; and Lighthouse accessibility rose from 94–97 to 100 after contrast and landmark fixes. Testing also exposed a broken object-level authorisation flaw, now fixed and covered by tests. Four of the five signed-in screens still use fixture data, which is the main work remaining.

Keywords: vehicle service management, single-page application, React, FastAPI, MongoDB, role-based access control, web accessibility

TABLE OF CONTENTS

Certificate ..... **ii**

Declaration ..... **iii**

Acknowledgement ..... **iv**

Abstract ..... **v**

List of Figures ..... **viii**

List of Tables ..... **ix**

2. INTRODUCTION **1**

2.1 Background 1

2.2 Motivation and relevance to the real-world problem 1

2.3 Scope and organisation of the report 2

3. PROBLEM STATEMENT AND OBJECTIVES **4**

3.1 Problem definition 4

3.2 Objectives 4

3.3 Constraints and assumptions 4

4. LITERATURE REVIEW **6**

4.1 Automobile Service Center Management System (Kawade et al., 2023) 6

4.2 Smart Vehicle Service Management System Using IoT (Allam et al., 2022) 6

4.3 Online Vehicle Service Management System (Kishor et al., 2026) 6

4.4 AutoServe – Automotive Service Management System (Jadhav et al., 2026) 6

4.5 Car Service Record System (Hairol Rizal and Omar, 2026) 7

4.6 Web-based Automobile Service Station Management System (Chandrasiri, 2022) 7

4.7 Tekmetric (commercial) 7

4.8 Shopmonkey (commercial) 7

4.9 Comparison and research gap 8

5. METHODOLOGY AND SYSTEM DESIGN **9**

5.1 Overall architecture 9
5.2 Flowchart: handling one request 10
5.3 Algorithms 10
5.4 Tools and technologies 12
5.5 Use-case and data-flow diagrams 12
6. IMPLEMENTATION 14
6.1 Client modules 14
6.2 Server modules 15
6.3 Dataset description 16
6.4 Pseudo code: ownership check 16
6.5 Screenshots 16
7. RESULTS AND DISCUSSION 20
7.1 Output tables: rules and authorisation 20
7.2 Performance graphs 21
7.3 Result interpretation 23
7.4 Comparison with objectives 23
8. CONCLUSION AND FUTURE SCOPE 24
8.1 Summary 24
8.2 Learning outcomes 24
8.3 Future enhancements 24
REFERENCES 25

LIST OF FIGURES

Figure 5.1: Overall architecture: browser SPA, Vite development server and proxy, FastAPI and MongoDB replica set 9
Figure 5.2: Flowchart of how the API handles one request 10
Figure 5.3: Use-case diagram for the four roles 12
Figure 5.4: Level-1 data-flow diagram 13
Figure 6.1: Public landing page and FAQ (static content) 17
Figure 6.2: Sign-in with click-to-fill demo accounts; customer view with two navigation links (fixture vehicles) 17
Figure 6.3: Manager dashboard on live API data in both themes 17
Figure 6.4: Staff screens: bookings calendar and job cards (fixture data) 18
Figure 6.5: Invoice with computed tax and total (fixture); the page a customer sees on a staff-only route 18
Figure 6.6: Part-issue failure notification (SIMULATED from fixture stock, not yet wired to the API's 409) 18
Figure 6.7: Responsive layouts at a 390-pixel phone width 19
Figure 6.8: Swagger UI authorised as the receptionist: an overlapping assignment is refused with 409 19
Figure 7.1: API response time per endpoint (live server, 60 requests each; login 20) 21
Figure 7.2: Time to fetch all seven dashboard reports: sequential versus parallel 21
Figure 7.3: Lighthouse accessibility score before and after the fixes 22
Figure 7.4: WCAG contrast ratio of each colour token before and after recalculation 22
Figure 7.5: Lighthouse performance and SEO scores after the fixes 22

LIST OF TABLES

Table 3.1: Objectives and how they are evaluated 4
Table 4.1: Comparison of related systems 8
Table 5.1: Tools and technologies 12
Table 6.1: Server modules 15
Table 6.2: Seeded dataset 16
Table 7.1: Business-rule and authorisation outcomes 20
Table 7.2: Outcomes against the objectives of Chapter 3 23

2. INTRODUCTION

2.1 Background

India has one of the largest vehicle populations in the world, and almost every one of those vehicles passes through a service centre several times a year. A typical centre is small to medium in size: a reception desk, a handful of service bays, six to twenty mechanics with different specialisations, a store of spare parts, and an owner who must keep all of it profitable. The work that flows through such a centre has a well-defined shape. A customer books a slot; the vehicle arrives and is inspected; a job card lists the tasks; each task is assigned to a mechanic for a window of time; parts are drawn from the store; the work is completed; an invoice is raised; and the customer pays and collects the vehicle.

In most centres each of these steps is recorded in a different place. Bookings are taken over the phone and written in a register. Job cards are printed sheets that travel with the vehicle. Mechanic allocation lives on a whiteboard. Parts are tracked in a spreadsheet, if at all. Invoices are typed into a billing program that knows nothing about the job card. The information exists, but it is not connected, and nobody outside the building can see it. The published systems reviewed in Chapter 4 describe the same starting point: manual registers and spreadsheets that lead to errors, delays and lost records [1, 2, 3].

The web is the natural medium for connecting these steps. A browser is already present on the reception computer, on the mechanic's phone and in the customer's pocket, so a web application can reach every participant without installing anything. Modern web technology makes this practical at small scale. Single-page applications (SPAs) render the interface in the browser and exchange only data with the server, which gives an application-like experience [4]. JSON APIs designed in the REST architectural style keep the server stateless and easy to test [5]. Document databases such as MongoDB now support multi-document ACID transactions, which makes them suitable for rules that must hold across several records [6].

This project, carried out for the Web Technologies course, applies these technologies to the service-centre workflow. The resulting application is called **AutoCare**. Its subject is the full stack of a modern web application: the client-side architecture, the API it consumes, the way identity and permission flow between them, and the quality attributes (accessibility, performance and correctness) that decide whether such an application is fit to be used by the public.

2.2 Motivation and relevance to the real-world problem

Three groups of people are poorly served by the manual process, and each motivates a different part of the system.

Customers cannot see what is happening to their car. They telephone to ask, they are told the vehicle is "being worked on", and the first detailed account they receive is the bill. Extra work is often discovered and carried out without a clear approval step, which is the most common source of disputes. A customer-facing portal that shows the job card task by task, and the invoice line by line, addresses this directly. Commercial products for large workshops already offer customer approval of

digital inspections over text message [7, 8]; small centres need the same transparency at much lower cost.

Staff lose time to coordination. A receptionist assigning a mechanic must remember who is free between 11:00 and 13:00; a double-booking is discovered only when two vehicles wait for the same person. A mechanic who needs a clutch plate finds out at the store that the last one was taken an hour ago. These are not interface problems; they are consistency problems, and they must be solved on the server, where concurrent requests meet. The system therefore treats its business rules (no overlapping assignments, no issue beyond stock, no client-supplied invoice totals) as the core of the design rather than as validation added to forms.

The owner has no single view. Revenue, mechanic workload, part consumption and turnaround time are all computable from the records a centre already keeps, but only if those records are connected. A dashboard that aggregates them in the database and presents them in one screen turns routine record-keeping into management information.

Beyond the business case, the project is relevant to the course because it exercises the full set of concerns a production web application must meet. Personal data (names, phone numbers, vehicle registrations, invoices) must be protected, and the most common web security failure category is broken access control [9]. The interface must be usable by everyone, including people with low vision, which the Web Content Accessibility Guidelines make measurable [10]. And the application must load quickly on a mid-range phone over a mobile connection, which tools such as Lighthouse make measurable [11]. Each of these is evaluated in Chapter 7 with numbers taken from the running system.

2.3 Scope and organisation of the report

In scope. The project covers four roles (customer, receptionist, mechanic, manager); customer registration and sign-in; vehicle and customer records; bookings; job cards with embedded tasks and inspections; mechanic assignment with the no-overlap rule; spare parts with the stock rule; part usage; invoices and payments with computed totals; and a reporting dashboard backed by aggregation pipelines. The deliverable is a working client and server, a seeded database, automated tests, and measurements of security, accessibility and performance.

Out of scope. Online payment-gateway integration, SMS or e-mail notifications, IoT diagnostics, multi-branch operation and production deployment are excluded. They are listed as future enhancements in Chapter 8.

Current state. The system runs end to end, but not every screen is finished. The manager dashboard reads live data from the API; the My garage, Bookings, Job cards and Invoice screens still render fixture data while the endpoints they need are completed. The report states this wherever it matters rather than presenting the prototype as complete.

The rest of the report is organised as follows:

- **Chapter 3** defines the problem precisely and sets measurable objectives, constraints and assumptions.

- **Chapter 4** reviews eight related systems, from published academic projects to commercial products, and compares them.
- **Chapter 5** presents the methodology and system design: architecture, request flowchart, the algorithms behind the business rules, the tools used, and use-case and data-flow diagrams.
- **Chapter 6** describes the implementation module by module, the seeded dataset, pseudo code and screenshots.
- **Chapter 7** reports results: test outcomes, latency, Lighthouse scores and contrast measurements, and compares them with the objectives.
- **Chapter 8** concludes with learning outcomes and future enhancements, followed by the references.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem definition

A service centre's records are fragmented across paper and disconnected software, so (i) customers cannot follow or approve the work on their vehicle, (ii) concurrent staff actions produce double-booked mechanics and over-issued parts, (iii) invoices are not reconciled with the work and parts actually recorded, and (iv) management cannot see revenue, workload or turnaround without manual compilation.

The problem for this project is therefore to design and implement a web application that keeps one consistent record of the service workflow; that lets each role see and change exactly the part of that record its job requires, and nothing more; that enforces the centre's business rules on the server so they hold under concurrent use; and that is accessible and fast enough to be used by the public on ordinary devices.

3.2 Objectives

Each objective is stated together with the evidence that will show whether it was met; Section 7.4 returns to this list.

Table 3.1: Objectives and how they are evaluated

#	Objective	Evidence of success
O1	Build a responsive single-page application with client-side routing and a consistent design system	7 routes; one token set for both themes; layouts verified at 390 px
O2	Authenticate users and authorise every request on the server by role and, for customers, by ownership	every endpoint guarded; a customer receives 404 for another customer's records; automated tests
O3	Enforce the three business rules under concurrent use	409/422 responses verified; concurrent stock test passes
O4	Drive screens from the JSON API with explicit loading, error and empty states	number of screens on live data
O5	Meet WCAG 2.1 AA contrast and structural requirements on public pages	Lighthouse accessibility score; contrast ratios $\geq 4.5 : 1$
O6	Keep the interface fast	API latency percentiles; Lighthouse performance; parallel vs sequential loading

Objectives O2 and O3 carry the most weight. An interface that hides a button from a customer but leaves the API open protects nothing, and a rule enforced only in a form can be bypassed by any request sent directly. Both objectives are therefore evaluated against the API, not only through the user interface.

3.3 Constraints and assumptions

Constraints

- **Team and time.** Four students over one semester, alongside other courses; the scope in Section 2.3 reflects this.

- **No paid services.** Everything runs on free, open-source software on a laptop; there is no cloud hosting, SMS gateway or payment gateway.
- **Transactions require a replica set.** MongoDB supports multi-document transactions only on a replica set, so even the development database runs as a single-node replica set [6].
- **Single origin in development.** The client and API run on different ports; a development proxy is used instead of opening the API to cross-origin requests.

Assumptions

- One service centre; multi-branch operation is out of scope.
- Each customer account is linked to exactly one customer record; staff accounts are created or promoted by the manager.
- Times are stored in UTC and assignment windows are half-open, so a window ending at 11:00 does not clash with one starting at 11:00.
- Prices are in Indian rupees and tax is applied at a single configured rate.
- The seeded dataset (Section 6.3) is representative in shape, but not in volume, of a small centre.

4. LITERATURE REVIEW

This chapter reviews eight related works: six academic projects that build service-centre or garage systems, and two commercial products used by repair workshops. Each is summarised from its own published description; where a publication does not mention a feature, the comparison in Section 4.9 records it as *not reported* rather than assuming it is absent.

4.1 Automobile Service Center Management System (Kawade et al., 2023)

Kawade and co-authors present a web-based platform for automobile service centres built with PHP and SQL [1]. It automates customer information management, vehicle tracking, inventory control and employee administration, and offers role-based access for administrators, staff and customers. Features include appointment scheduling, repair status updates, payment processing and service reminders, presented through a central dashboard. The work demonstrates the breadth of functions a centre needs and the value of giving customers a view of repair status. The publication describes the features but not how conflicting actions (two staff scheduling the same resource, or two issues of the last part) are prevented, and reports no evaluation of security, accessibility or performance. For AutoCare, the lesson is that a feature list is the easy half of the problem: each feature that lets two people act on the same record also needs a rule for what happens when they act at once.

4.2 Smart Vehicle Service Management System Using IoT (Allam et al., 2022)

Allam et al. take a different angle: sensors installed in the vehicle detect problems, the system sends SMS alerts to the owner, and an online platform lets the owner book a service [12]. Force-sensitive, accelerometer and temperature/humidity sensors provide the inputs. The contribution is in *triggering* a service visit at the right time rather than in managing the workshop once the vehicle arrives. It is complementary to AutoCare, which starts at the booking: sensor-triggered booking is a natural future integration, and Chapter 8 lists it as such. The paper is also a reminder that customers value being told *before* something goes wrong, which supports AutoCare's emphasis on visible job-card progress.

4.3 Online Vehicle Service Management System (Kishor et al., 2026)

Kishor et al. describe a web application built with PHP on a XAMPP server for a service centre that is busy during festive seasons [13]. Its distinctive feature is workload distribution: repair tasks are allocated to staff according to booking time and priority, and the system also allocates doorstep vehicle pick-up to staff. This is the closest of the reviewed works to AutoCare's mechanic assignment, but the paper describes allocation rather than conflict prevention: it does not state what happens if a staff member is given two overlapping tasks. AutoCare makes that case explicit: the overlap test is a single indexed query run in the same transaction as the insert, and an overlapping request is refused with the identity of the conflicting assignment so the receptionist can choose another slot.

4.4 AutoServe – Automotive Service Management System (Jadhav et al., 2026)

AutoServe is the most technically similar system reviewed [3]. It is a web application built with Next.js, TypeScript and MySQL, with authentication delegated to a third-party service (Clerk). Customer data, service history, technician work, inventory and billing are handled in one place, and a dashboard shows real-time updates on services, stock levels and performance. Like AutoCare it uses a

typed, component-based frontend and a dashboard for the owner. It differs in its relational database and in outsourcing identity, whereas AutoCare implements tokens and role checks itself, which made the authorisation flaw discussed in Section 7.1 both possible and visible. Delegating sign-in to a provider settles *who* the caller is; it does not settle *which records* that caller may read, which every application must still check for itself.

4.5 Car Service Record System (Hairol Rizal and Omar, 2026)

Hairol Rizal and Omar developed a web-based car service recording system for workshops that rely on manual methods, following the waterfall model and using XAMPP as the local server [2]. The system organises bookings, service records and payments in a structured data hierarchy, and the authors report that it addresses misplaced documents, scheduling conflicts and difficulty in retrieving past records. Its value for this project is the clear statement of the problem from the workshop's side, which matches the background in Chapter 2. Its local XAMPP stack and waterfall process also illustrate a common trade-off in student projects: a simple, familiar stack in exchange for transactions, schema validation and automated tests, on which AutoCare relies.

4.6 Web-based Automobile Service Station Management System (Chandrasiri, 2022)

Chandrasiri's master's thesis presents a web platform for automobile service stations in which customers manage reservations, view vehicle maintenance history and purchase parts online, while management generates administrative reports and maintains inventory [14]. The emphasis on maintenance history as a customer-facing asset is shared with AutoCare's My garage screen. The thesis frames the system in terms of service quality and reduced delays rather than data consistency. Allowing customers to buy parts online would add a second path into the parts store, which is exactly the situation in which AutoCare's atomic stock rule matters most.

4.7 Tekmetric (commercial)

Tekmetric is a cloud-based shop management product for repair workshops [7]. Its public description emphasises digital vehicle inspections whose results are sent to the customer by text or e-mail for approval, estimates built from vehicle-specific labour times, parts ordering with automatic inventory updates, online scheduling, payment processing including text-to-pay, two-way texting, and real-time reporting across multiple shops. It represents the commercial state of the art in customer transparency, particularly the approval step for extra work, which AutoCare's design anticipates but has not yet built. It also shows how much of the value lies in communication (texts, approvals, reminders) rather than in record-keeping alone.

4.8 Shopmonkey (commercial)

Shopmonkey is a second cloud-based workshop product [8]. It offers real-time workflow management, digital inspections sent by e-mail or text, digital estimates and invoices, online and in-person payments, appointment confirmations and reminders, built-in messaging, reporting and accounting. Commercial products such as these confirm which features customers and owners value. Being proprietary, they do not publish how they enforce scheduling or stock consistency, so they cannot be compared on those points. Their price and breadth are aimed at established workshops; the

gap AutoCare targets is a smaller centre that needs the same consistency and transparency in a system it can understand and run itself.

4.9 Comparison and research gap

Table 4.1: Comparison of related systems

Work	Stack	Online booking	Customer sees progress	Inventory	Billing	Role-based access	Conflict rules stated	Security / a11y evaluated
Kawade 2023	PHP, SQL	Yes	Yes	Yes	Yes	Yes	Not reported	Not reported
Allam 2022	IoT + web	Yes	alerts	Not reported	Not reported	Not reported	Not reported	Not reported
Kishor 2026	PHP, XAMPP	Yes	Not reported	Not reported	Not reported	staff	allocation only	Not reported
Jadhav 2026	Next.js, MySQL	Not reported	Not reported	Yes	Yes	via Clerk	Not reported	Not reported
Rizal 2026	PHP, XAMPP	Yes	Not reported	Not reported	Yes	Not reported	Not reported	Not reported
Chandrasiri 2022	Web	Yes	history	Yes	Not reported	Not reported	Not reported	Not reported
Tekmetric	Cloud SaaS	Yes	inspections + approval	Yes	Yes	Yes	proprietary	proprietary
Shopmonkey	Cloud SaaS	Yes	inspections	Yes	Yes	Yes	proprietary	proprietary
AutoCare (this work)	React, FastAPI, MongoDB	API built; screen on fixtures	screen on fixtures	Yes	Yes	role + ownership	**overlap, stock, totals in transactions**	**tests, Lighthouse, WCAG contrast**

Two patterns stand out. First, the academic systems describe *features*, but none of the publications states how conflicting actions are prevented, for example two receptionists assigning one mechanic to overlapping windows, or two mechanics taking the last unit of a part. These are exactly the failures that make paper systems unreliable, and they are not solved by moving the same read-then-write logic onto a web page. Second, none of the academic works evaluates security or accessibility. Broken access control is the most common category of web application vulnerability [9], and broken object-level authorisation is ranked first among API risks [15].

AutoCare is designed to address this gap: it states each business rule, enforces it on the server inside a transaction, and tests it, including under concurrency; and it evaluates authorisation, accessibility and performance with reproducible measurements. Its weakness relative to the reviewed systems is completeness of the customer-facing screens, which is reported honestly in Chapter 7.

5. METHODOLOGY AND SYSTEM DESIGN

The project followed an iterative process. The data model and business rules were designed first, implemented as an API with tests, and only then consumed by the interface. Each iteration ended with measurement: tests, a browser check of every route per role, and Lighthouse audits. Several defects reported in Chapter 7 were found this way rather than by inspection.

5.1 Overall architecture

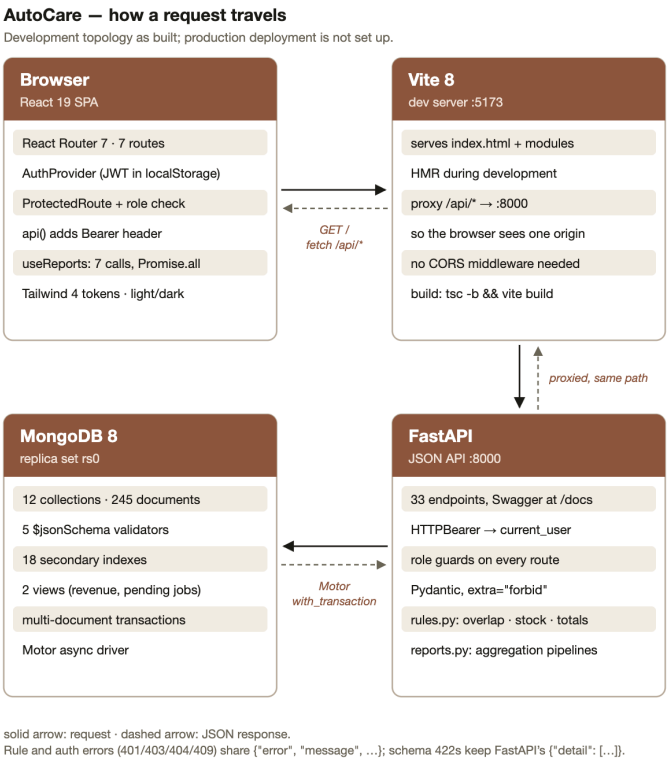


Figure 5.1: Overall architecture: browser SPA, Vite development server and proxy, FastAPI and MongoDB replica set

The system has three tiers. The **client** is a React single-page application [16]: the browser downloads it once and then exchanges only JSON with the server. In development the **Vite** server serves the application with hot module replacement and forwards every request under `/api` to the API on port 8000 [17]. Because the browser only ever talks to one origin, the API needs no cross-origin (CORS) configuration, which removes a common source of misconfiguration. The **server** is a FastAPI application [18] that validates every request body against a schema, authenticates the caller, applies role and ownership checks, and runs business rules inside MongoDB transactions [6]. The **database** is MongoDB 8 running as a single-node replica set, which is the minimum configuration that supports multi-document transactions. The API is stateless in the REST sense [5]: every request carries a bearer token, and no session is held on the server.

5.2 Flowchart: handling one request



Figure 5.2: Flowchart of how the API handles one request

Every protected request passes the same sequence of gates (Figure 5.2). The order matters. Authentication comes first, so an anonymous caller learns nothing. The user is then re-read from the database, so a deleted account or a changed role takes effect immediately instead of when the token expires. The role guard comes next, then the ownership check for customers, and only then are the body validated and the business rule evaluated inside a transaction. A failure at any gate returns a specific status code (401, 403, 404, 409 or 422) with a machine-readable error body, and a failure inside the transaction rolls back every write made by that request.

5.3 Algorithms

No overlapping assignments (half-open intervals)

Two time windows $[s_1, e_1)$ and $[s_2, e_2)$ overlap exactly when $s_1 < e_2$ and $e_1 > s_2$. Using strict inequalities makes the windows half-open, so an assignment ending at 11:00 and the next starting at 11:00 are legal; non-strict comparisons would wrongly reject that common case. The check is a single indexed query on the `assignments` collection, filtered by mechanic and excluding cancelled assignments. A compound index on (mechanic_id, start_time, end_time) serves it. Because the query and the insert run inside one transaction, two receptionists assigning the same mechanic at once cannot both succeed.

```
function create_assignment(job_card, task, mechanic, start, end):
    begin transaction
        require job_card exists and task belongs to it        -- else 404
        clash ← find one assignment where mechanic = mechanic
                  and status ≠ cancelled and start_time < end and end_time > start
        if clash exists: abort, 409 (return clash id and window)
        insert assignment(job_card, task, mechanic, start, end, status = scheduled)
    commit
```

Pseudo code 5.1: mechanic assignment with the overlap rule

Issuing a part without overselling stock

The naive approach reads the stock level, checks it, and then writes the new level. Between the read and the write another request can take the same units, and both succeed. AutoCare instead places the condition inside the update itself: the filter matches the part only if `stock_qty ≥ qty`, and the update decrements it atomically. If two requests race for the last unit, the second matches no document, modifies nothing and is rejected with 409. The part-usage record is written in the same transaction, so stock and usage cannot disagree. An automated test fires two concurrent requests for the last unit and asserts that exactly one succeeds.

Computed invoice totals

An invoice is built on the server from the part-usage records of the job card plus labour lines entered by the receptionist; the subtotal, tax and total are computed by a pure function. The request schema forbids extra fields, so a client that sends its own `total` receives 422 instead of having it silently ignored or, worse, stored.

Authentication, role guards and ownership

On sign-in, the password is checked against a bcrypt hash and the server issues a signed JSON Web Token carrying the user id and an expiry 12 hours ahead [19]. Every protected route declares a named guard (`staff_only`, `front_desk`, `workshop`, `manager_only` or `current_user`) listing the roles allowed. Roles alone are not enough: a customer is allowed to read job cards, but only *their own*. The ownership check resolves the owner of the requested record (job card → booking → customer, or vehicle → customer) and compares it with the caller's customer id. On mismatch it returns 404, identical to a missing record, so a customer cannot even confirm that another customer's record exists. This is the standard mitigation for broken object-level authorisation [15].

Loading the dashboard in parallel

The dashboard needs seven independent reports. Awaiting them one after another makes the page as slow as their *sum*; issuing them together with `Promise.all` makes it as slow as the *slowest*. A cancellation flag discards responses that arrive after the user has left the page. Section 7.2 measures the difference.

5.4 Tools and technologies

Table 5.1: Tools and technologies

Layer	Technology (version)	Why it was chosen
UI library	React 19.2.8 + TypeScript 6.0.3	component model; types catch API-shape mistakes at build time [16]
Routing	React Router 7.18.2	nested layout routes and route guards
Styling	Tailwind CSS 4.3.3	utility classes; design tokens as CSS custom properties [20]
Motion	Framer Motion 13.1.1, Lenis 1.3.26	enter/exit animation; smooth scrolling on the landing page
Accessible primitives	Radix UI dialog, label, slot	focus trapping and ARIA handled by tested components
Build / dev server	Vite 8.2.2	fast hot reload; `/api` proxy; production bundling [17]
API framework	FastAPI + Pydantic (Python 3.12)	schema validation, dependency-injected guards, generated Swagger UI [18]
Database	MongoDB 8.3 (replica set) + Motor	documents fit job cards with embedded tasks; transactions on a replica set [6]
Security	bcrypt, python-jose (JWT)	salted password hashes; signed, expiring tokens [19]
Quality	pytest, oxlint, Lighthouse 13.5	28 automated tests; linting; accessibility and performance audits [11]

5.5 Use-case and data-flow diagrams

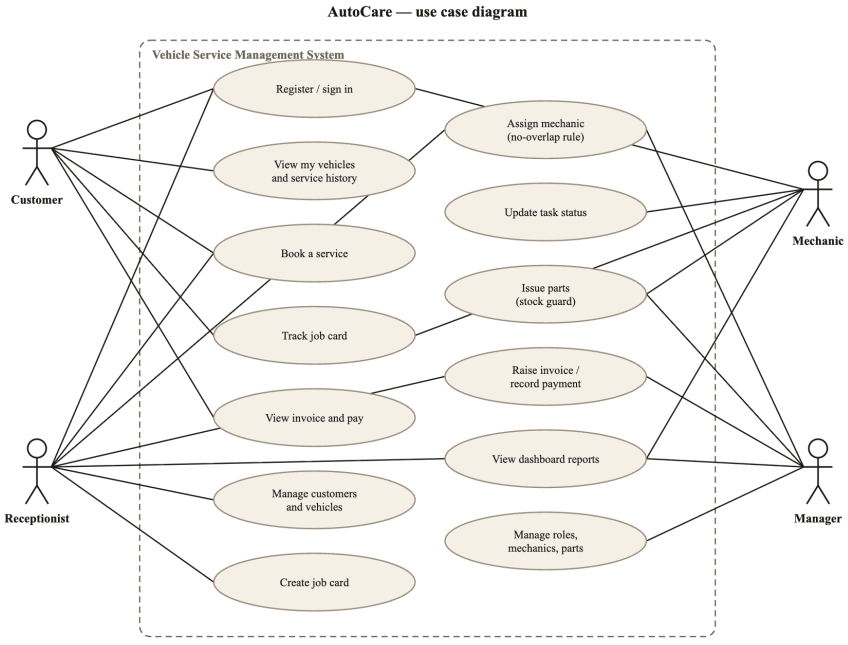


Figure 5.3: Use-case diagram for the four roles

Figure 5.3 shows the use cases per role. The customer's use cases are all about *their own* records, which is why ownership, not only role, must be checked. The receptionist creates the records that start

the workflow; the mechanic changes task status and issues parts; the manager sees the reports and manages roles, mechanics and the parts catalogue.

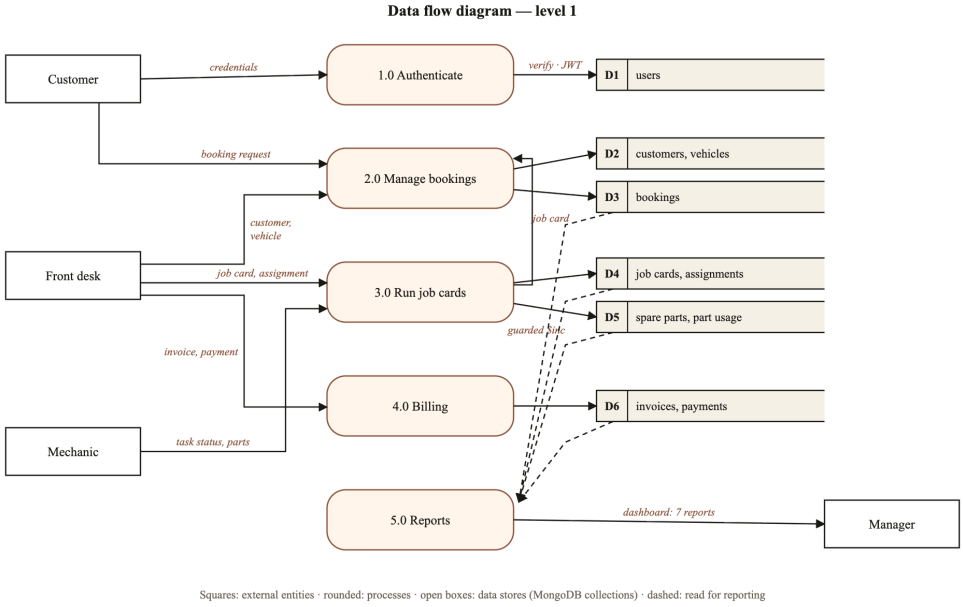


Figure 5.4: Level-1 data-flow diagram

Figure 5.4 traces data through five processes. Bookings, job cards and billing each write to their own collections, and the reporting process reads all of them through aggregation pipelines rather than duplicating data into separate report tables.

6. IMPLEMENTATION

The implementation consists of a client of 3,433 lines of TypeScript, TSX and CSS in 30 files, and a Python API with 33 endpoints. This chapter describes it module by module, then the dataset, pseudo code and screens.

6.1 Client modules

Routing and layout

`App.tsx` declares seven routes. Public routes are the landing page (`/`) and sign-in (`/auth`). Everything under `/app` is wrapped in a `ProtectedRoute` that requires a session, and in an `AppShell` layout that draws the navigation; the child page renders into the layout's outlet. A second `ProtectedRoute` with a `roles` list wraps the staff pages (dashboard, bookings) and the workshop page (job cards). The component's own comment records the key design principle: it is a convenience for the user, not a security boundary.

```
<Route element={<ProtectedRoute />}>                {/* any signed-in user */}
  <Route path="/app" element={<AppShell />}>
    <Route index element={<MyGarage />} />
    <Route path="invoice" element={<Invoice />} />
    <Route element={<ProtectedRoute roles={STAFF} />}>    {/* receptionist,
mechanic, manager */}
      <Route path="dashboard" element={<Dashboard />} />
      <Route path="bookings" element={<Bookings />} />
    </Route>
    <Route element={<ProtectedRoute roles={WORKSHOP} />}> {/* mechanic, manager */}
      <Route path="jobs" element={<Jobs />} />
    </Route>
  </Route>
</Route>
```

Listing 6.1: nested routes in App.tsx (condensed)

Authentication state and the API client

`AuthProvider` is a React context holding the signed-in user. On load it calls `/api/auth/me` if a token is stored; on sign-in it stores the token returned by `/api/auth/login`. All requests go through one function, `api()`, which attaches the bearer token, removes the token on any 401 so that an expired session returns the user to sign-in instead of looping, and converts error bodies into an `ApiError`. The server returns two error shapes, its own `{error, message}` and FastAPI's validation shape `{detail: [...]}`, and `api()` reads both. The token is kept in `localStorage`, which is simple but readable by any injected script; OWASP advises against storing session identifiers there [21], and Chapter 8 lists the move to an httpOnly cookie.

Data fetching

`useReports()` issues the seven dashboard requests together with `Promise.all`, keeps `loading`, `error` and `data` state, and ignores late responses after unmount. Every panel has three explicit states: a shimmering skeleton while loading, the server's own message on error, and a per-panel empty state when the database has no data, so an empty database renders zeros instead of `NaN`.

Design system

The visual language is a warm-neutral neomorphism: controls appear raised from or pressed into the page through a pair of shadows, one light and one dark. All colours are semantic tokens declared once as CSS custom properties (`--background`, `--card`, `--accent`, `--destructive` and so on) and mapped into Tailwind with `@theme inline`. Dark mode re-declares the same tokens under a `.dark` class and inverts the shadow pair, so no component contains a dark-mode rule of its own. A `prefers-reduced-motion` media query cuts all animation to near zero for users who ask for it. The token values were later recalculated for contrast (Section 7.2).

6.2 Server modules

The API is organised into three routers (`auth`, `core`, `reports`), a service layer holding the business rules, and a setup script that creates validators, indexes and views.

Table 6.1: Server modules

Module	Responsibility	Key details
app/auth.py	passwords, tokens, guards	bcrypt hashing capped at 72 bytes; JWT (HS256, 12 h); `HTTPBearer` scheme so Swagger UI can authorise; named role guards
app/routers/auth.py	register, login, me, user roles	public registration always creates a customer; identical message for unknown e-mail and wrong password
app/routers/core.py	customers, vehicles, bookings, job cards, assignments, parts, invoices, payments	ownership check for customer reads and bookings (404 on mismatch)
app/services/rules.py	the three business rules	overlap query; guarded `$inc`; computed totals; each in a transaction
app/services/reports.py	eight aggregation pipelines	`$match`, `$group`, `$lookup`, `$project`; summary and centre-health indicators
app/errors.py	domain errors → HTTP	one JSON shape: error code, message, details
scripts/init_indexes.py	schema setup	5 `$jsonSchema` validators, 18 secondary indexes, 2 views; idempotent

Of the 33 endpoints, 18 are GET, 12 are POST and 3 are PATCH. Their guards are distributed as follows: `staff_only` 13; `front_desk` 6; `workshop` 2; `manager_only` 4; `current_user` 5; `public` 3. The full list is generated by the framework and browsable in Swagger UI at `/docs`.

```
db.assignments.aggregate([
  { $match: { status: { $ne: "cancelled" } } },
  { $group: { _id: "$mechanic_id",
              total_hours: { $sum: { $divide: [ { $subtract: ["$end_time",
"$start_time"] }, 3600000 ] } },
              jobs_count:  { $sum: 1 } } },
  { $lookup: { from: "mechanics", localField: "_id", foreignField: "_id", as:
"mechanic" } },
  { $unwind: "$mechanic" },
  { $sort: { total_hours: -1 } } ])
```

Listing 6.2: mechanic-workload report as an aggregation pipeline (condensed)

6.3 Dataset description

A deterministic seed script loads a realistic small-centre dataset: the same documents are produced on every run, so tests, screenshots and measurements are reproducible. Totals and settlement dates are computed by the same functions the API uses, so the seed cannot contradict the rules.

Table 6.2: Seeded dataset

Collection	Documents	Validator	Secondary indexes	Contents
customers	15		1	name, phone, e-mail
vehicles	20	yes	2	registration (unique, normalised), make, model, year, owner
bookings	25	yes	2	vehicle, customer, service type, dates, status
job_cards	20		2	booking, embedded tasks, status
inspections	20		1	40-point checks and advisories per job card
mechanics	6		0	name, specialisation
assignments	32	yes	2	mechanic, task, start/end window, status
spare_parts	20	yes	1	SKU (unique), stock, reorder level, price
part_usage	40		2	job card, part, quantity, line total
invoices	20	yes	3	line items, subtotal, tax, total
payments	23		1	invoice, amount, method
users	4		1	e-mail (unique), bcrypt hash, role, customer link
Total	**245**	**5**	**18**	plus 2 views: revenue by month, pending jobs

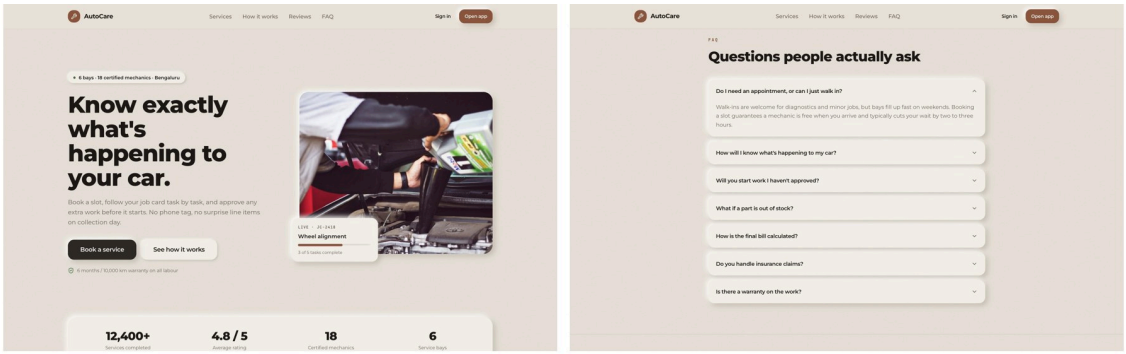
6.4 Pseudo code: ownership check

```
function visible_job_card(user, job_card_id):
    job_card ← find job_cards by id                -- 404 "job_card not found" if
missing
    if user.role = customer:
        booking ← find bookings by job_card.booking_id
        if booking is null or booking.customer_id ≠ user.customer_id:
            raise 404 "job_card not found"         -- same status and body as a
missing record
    return job_card                                -- staff roles see every
customer's records
```

Pseudo code 6.1: ownership check used by the job-card and inspection endpoints

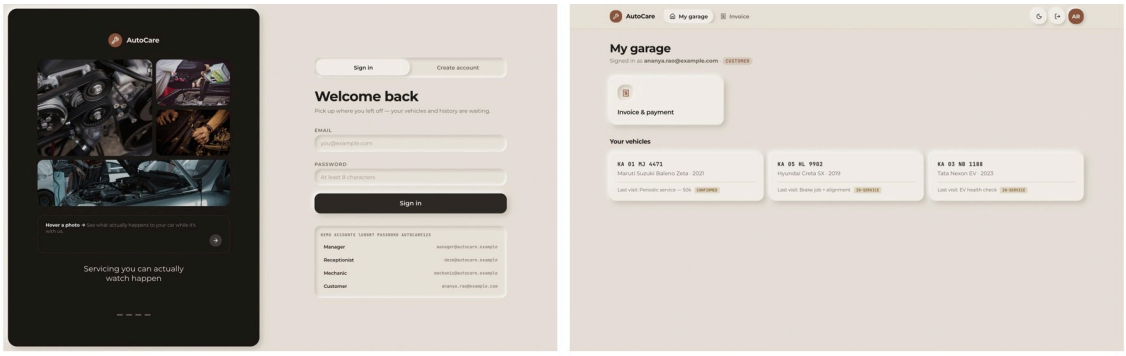
6.5 Screenshots

All screenshots were taken from the running application by a scripted browser signed in as the role named. Fixture-driven screens are labelled as such.



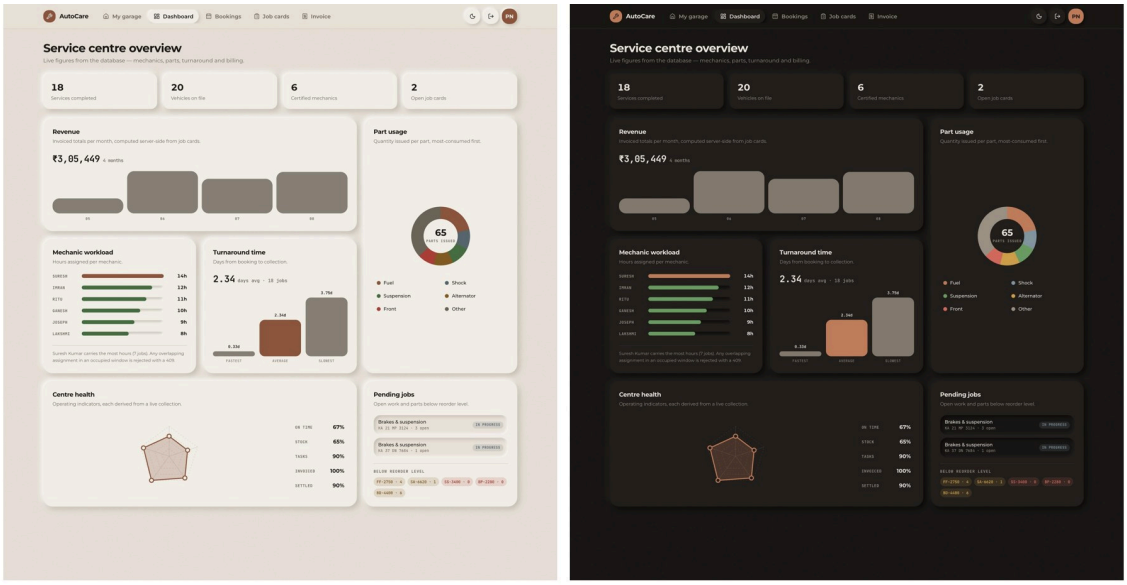
(a) Landing page (b) FAQ section

Figure 6.1: Public landing page and FAQ (static content)



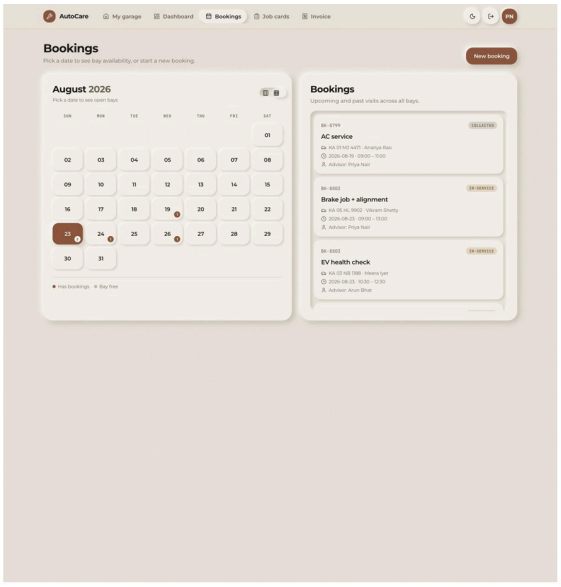
(a) Sign in (b) Customer: My garage

Figure 6.2: Sign-in with click-to-fill demo accounts; customer view with two navigation links (fixture vehicles)

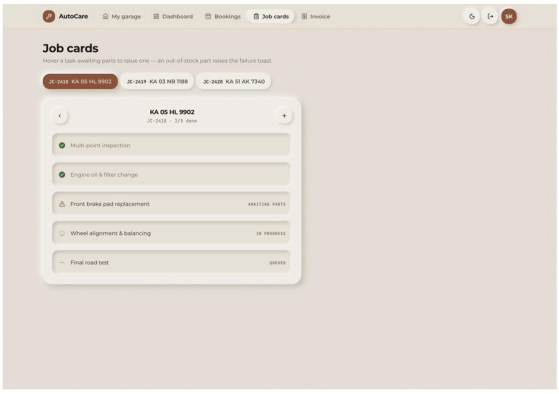


(a) Light theme (b) Dark theme

Figure 6.3: Manager dashboard on live API data in both themes

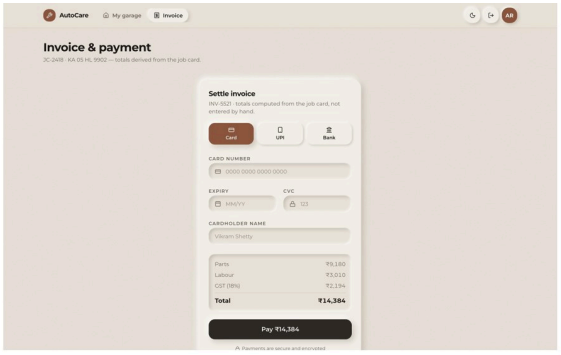


(a) Bookings calendar

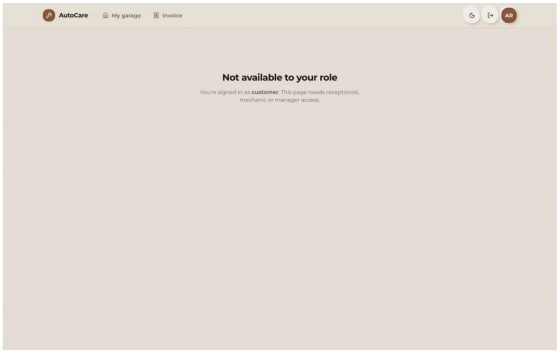


(b) Job cards

Figure 6.4: Staff screens: bookings calendar and job cards (fixture data)



(a) Invoice and payment



(b) Role page for a customer

Figure 6.5: Invoice with computed tax and total (fixture); the page a customer sees on a staff-only route

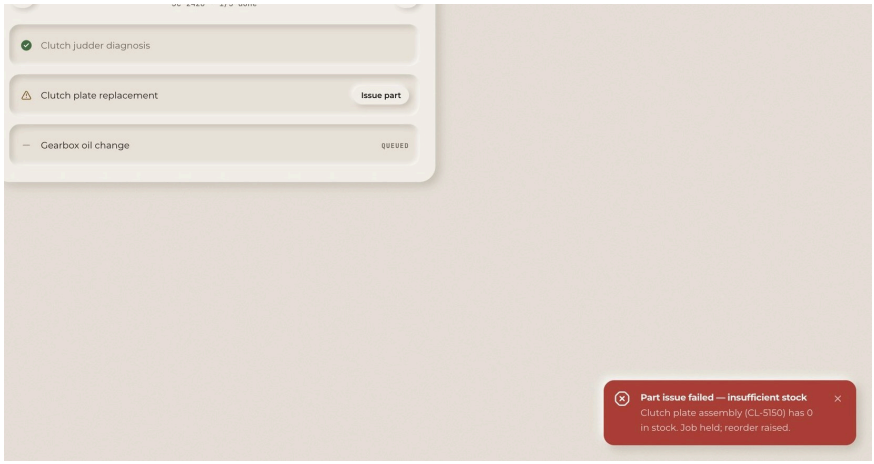
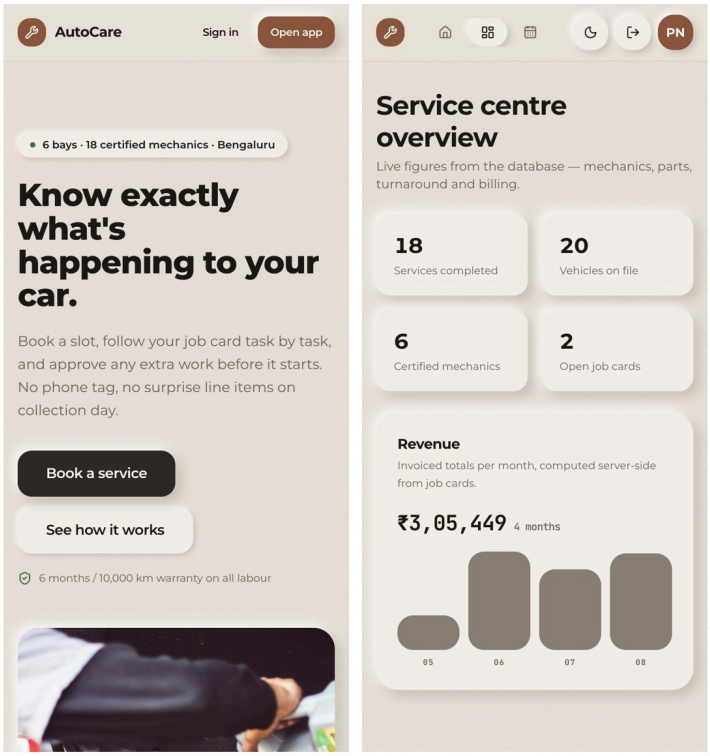


Figure 6.6: Part-issue failure notification (SIMULATED from fixture stock, not yet wired to the API's 409)



(a) Landing at 390 px (b) Dashboard at 390 px

Figure 6.7: Responsive layouts at a 390-pixel phone width

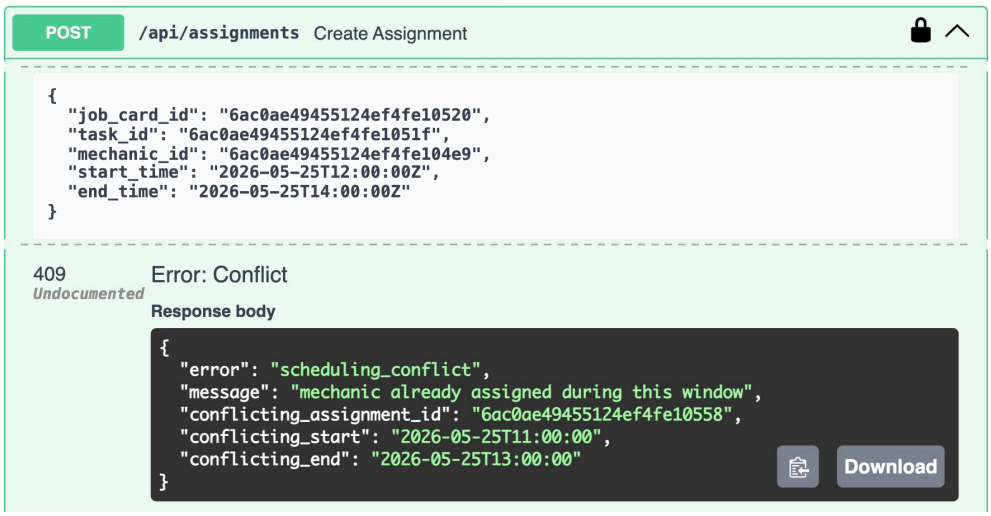


Figure 6.8: Swagger UI authorised as the receptionist: an overlapping assignment is refused with 409

7. RESULTS AND DISCUSSION

7.1 Output tables: rules and authorisation

Table 7.1: Business-rule and authorisation outcomes

Request (sent to the live API)	Expected	Observed
Assign a mechanic to a window overlapping an existing assignment	409, names the clash	409 · assignment count unchanged (32)
Assign a window that starts exactly when another ends	201	201 (automated test)
Issue more units of a part than are in stock	409 with SKU and stock	409 (automated test)
Two concurrent requests for the last unit of a part	exactly one succeeds	one 201, one 409 (automated test)
Invoice body containing a client-supplied `total`	422	422
Any protected endpoint with no token / a forged token	401	401 / 401
Customer token on the customer list	403	403
Customer token on another customer's job card, inspections, vehicle	404	**200 before the fix; 404 after**
Customer booking a service on another customer's vehicle	404, nothing written	404 · booking not written (automated test)

The automated suite has 28 tests: 12 for passwords, tokens and role guards; 9 for the business rules against a real MongoDB replica set; and 7 for ownership. All 28 pass. The ownership tests were written to fail first: run against the code as it was before the fix, 5 of the 7 failed, which shows that they detect the flaw rather than merely passing.

The most important result in this table is the row that changed. Every endpoint had a role guard, and the interface never showed a customer another customer's data, yet a customer token could read any job card, its inspection report and any vehicle (including the owner's id) simply by changing an identifier in the URL. This is broken object-level authorisation, the first-ranked API risk [15]. It was found by testing the API directly rather than through the interface, fixed by the ownership check in Section 5.3, and verified both by the tests and by repeating the original requests against the live server.

7.2 Performance graphs

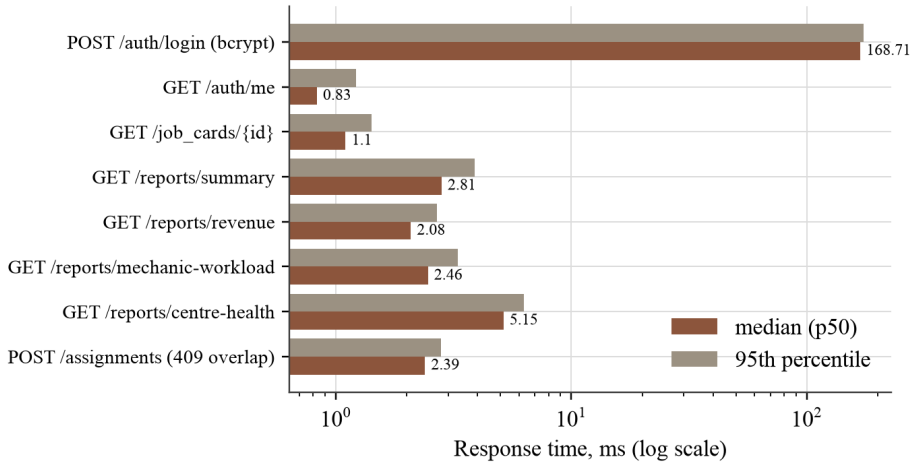


Figure 7.1: API response time per endpoint (live server, 60 requests each; login 20)

Every read endpoint answers in a median of under 6 ms, including the aggregation reports (Figure 7.1). The slowest call by two orders of magnitude is sign-in, at 169 ms. This is by design: bcrypt is deliberately slow so that stolen password hashes are expensive to attack, and sign-in happens once per session. The refused overlapping assignment costs 2.39 ms, so the rule adds no noticeable delay.

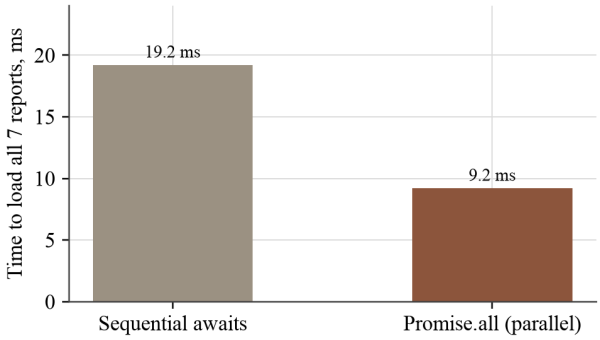


Figure 7.2: Time to fetch all seven dashboard reports: sequential versus parallel

Loading the seven dashboard reports in parallel took a median of 9.2 ms against 19.2 ms sequentially over 25 runs (Figure 7.2), a 2.1× improvement on a local network. Over a real mobile connection, where each request also pays network latency, the gap would be larger, since parallel requests pay it roughly once.

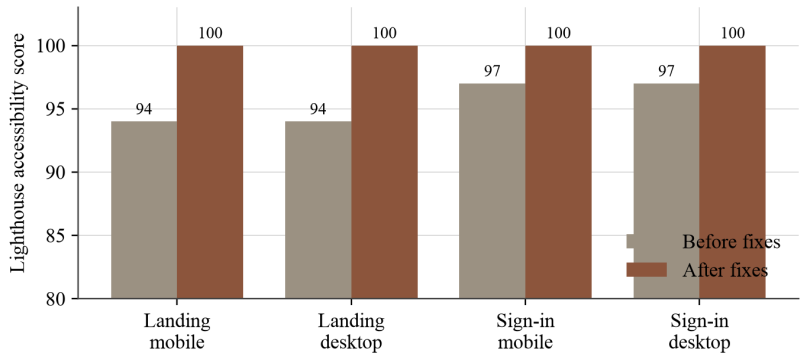


Figure 7.3: Lighthouse accessibility score before and after the fixes

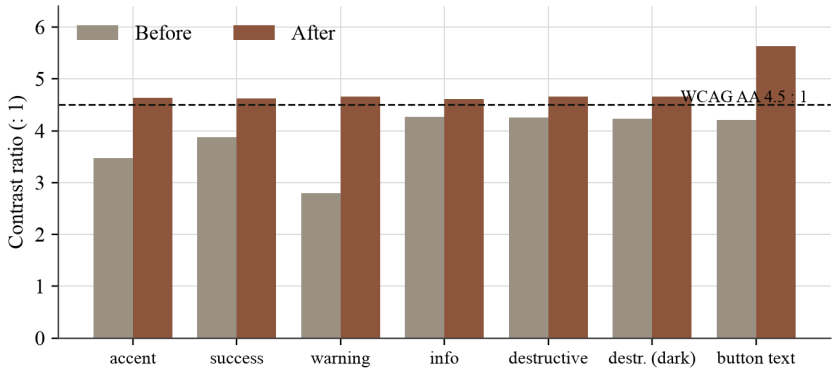


Figure 7.4: WCAG contrast ratio of each colour token before and after recalculation

Lighthouse found two accessibility failures on the public pages: text contrast below the WCAG AA threshold of 4.5 : 1, and no `<main>` landmark [10]. Rather than fix only the elements Lighthouse sampled, every semantic colour token was recalculated with the WCAG formula against both the page and card backgrounds (Figure 7.4). Five text colours and the button text failed; all now pass. Chart bars, which carry data and therefore need 3 : 1, rose from 1.31 : 1 to 3.39 : 1 in the light theme. Accessibility is now 100 on all four audited runs (Figure 7.3).

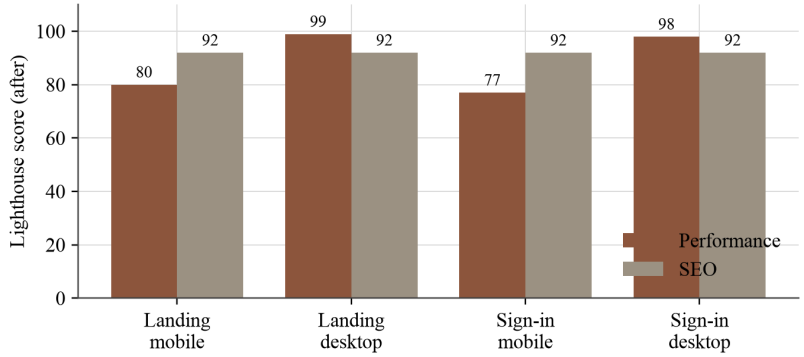


Figure 7.5: Lighthouse performance and SEO scores after the fixes

Performance is 98–99 on desktop but 77–80 on the throttled mobile profile, with a largest contentful paint of about 5 s (Figure 7.5). Two causes are visible in the audits: the whole application ships as one 533 kB JavaScript bundle (166 kB compressed), so the landing page downloads the dashboard's code

too; and the sign-in page loads four 900-pixel remote photographs. Route-level code splitting and correctly sized local images are the remedies. The only SEO failure is a missing `robots.txt`.

7.3 Result interpretation

The server-side results are strong: every business rule and authorisation path behaves as specified, under test and on the live server, with negligible latency cost. The results also show the value of testing at the API boundary. The interface had looked correct throughout, and the authorisation flaw was invisible until requests were sent directly. The client-side results are mixed: accessibility and responsiveness meet their targets, but mobile load performance does not yet, and most signed-in screens are not yet on live data.

Three further defects were found during the work and fixed: a payment screen that never advanced because a state change waited on an exit animation; dashboard bar charts that drew an 82% bar the same height as a 100% bar because of flex-box shrinking (found only by measuring rendered pixels); and a missing security scheme that prevented Swagger UI from authorising requests.

7.4 Comparison with objectives

Table 7.2: Outcomes against the objectives of Chapter 3

#	Objective	Outcome	Evidence
O1	Responsive SPA, routing, design system	Met	7 routes; one token set, two themes; 390 px layouts (Figure 6.7)
O2	Authorise by role and ownership on the server	Met (after fix)	guards on all 33 endpoints; ownership 404; 7 tests
O3	Business rules under concurrency	Met	409/422 outcomes; concurrent stock test passes
O4	Screens driven by the API	Partial	1 of 5 signed-in screens live; three-state handling built
O5	WCAG AA on public pages	Met	Lighthouse accessibility 100; all tokens ≥ 4.5 : 1
O6	Fast interface	Partial	API medians < 6 ms; dashboard 2.1× faster in parallel; mobile Lighthouse 77–80

8. CONCLUSION AND FUTURE SCOPE

8.1 Summary

This project built AutoCare, a web-based Vehicle Service Management System that connects booking, job cards, mechanic assignment, parts, billing and reporting in one record. The server enforces the centre's business rules inside transactions and authorises every request by role and ownership; the client is an accessible single-page application with a reusable design system. Four of six objectives are fully met and two partly, and every claim in this report is backed by a test or a measurement.

8.2 Learning outcomes

- A route guard in the browser is not security; authorisation must be enforced, and tested, at the API.
- Correctness under concurrency comes from atomic operations and transactions, not from checking before writing.
- Accessibility and performance are measurable, and measuring them changes the design.
- Reading code is not verification: two of the defects found looked correct in the source.

8.3 Future enhancements

1. Wire the My garage, Bookings, Job cards and Invoice screens to the API, and replace the simulated part-failure notification with the real 409.
2. Build screens for vehicle registration, mechanic assignment and parts management, which the API already supports.
3. Add customer approval of additional work, as offered by commercial products [7].
4. Move the session token to an httpOnly cookie and distinguish "offline" from "signed out" on start-up.
5. Apply route-level code splitting and serve images at display size to raise mobile performance.
6. Integrate SMS reminders and sensor-triggered booking along the lines of [12], and deploy to a single public origin.

REFERENCES

[1] V. Kawade, S. Wankhade, M. Raut, S. Pawar, A. Dhoke, N. Dafade, and A. B. Deshmukh, "Automobile service center management system," *Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET)*, May 2023, doi: 10.22214/ijraset.2023.52141.

[2] M. I. Hairol Rizal and N. A. Omar, "Car service record system," *Applied Information Technology and Computer Science*, vol. 7, no. 1, pp. 352–370, 2026, doi: 10.30880/aitcs.2026.07.01.019.

[3] D. Jadhav, S. A. Pawar, N. K. Pawar, and V. J. Mane, "AutoServe – automotive service management system," *Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET)*, vol. 14, no. 3, Mar. 2026, doi: 10.22214/ijraset.2026.78545.

[4] A. Mesbah and A. van Deursen, "Migrating multi-page web applications to single-page Ajax interfaces," in *Proc. 11th European Conf. Software Maintenance and Reengineering (CSMR)*, 2007, pp. 181–190.

[5] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.

[6] MongoDB, Inc., "Transactions," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/ (Accessed: Oct. 5, 2026).

[7] Tekmetric, "Auto repair shop management software." [Online]. Available: https://www.tekmetric.com/ (Accessed: Oct. 5, 2026).

[8] Shopmonkey, "Auto repair shop software." [Online]. Available: https://www.shopmonkey.io/ (Accessed: Oct. 5, 2026).

[9] OWASP Foundation, "A01:2021 – Broken Access Control," OWASP Top 10, 2021. [Online]. Available: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ (Accessed: Oct. 5, 2026).

[10] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/

[11] Google, "Lighthouse overview," Chrome for Developers. [Online]. Available: https://developer.chrome.com/docs/lighthouse/overview (Accessed: Oct. 5, 2026).

[12] M. S. Allam, M. Iyead, P. Sagar, and Rajatha, "Smart vehicle service management system using IoT," *Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET)*, vol. 10, no. 6, Jun. 2022, doi: 10.22214/ijraset.2022.44767.

[13] B. A. Kishor, F. A. Parmeshwar, J. P. Ganesh, and P. Khandekar, "Online vehicle service management system," *Int. J. Res. Appl. Sci. Eng. Technol. (IJRASET)*, vol. 14, no. 3, Mar. 2026, doi: 10.22214/ijraset.2026.78179.

[14] R. V. R. Chandrasiri, "Web-based automobile service station management system," M.I.T. thesis, University of Colombo School of Computing, Colombo, Sri Lanka, 2022.

[15] OWASP Foundation, "OWASP API Security Top 10 – 2023: API1 Broken Object Level Authorization." [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (Accessed: Oct. 5, 2026).

[16] Meta Open Source, “React documentation.” [Online]. Available: https://react.dev/ (Accessed: Oct. 5, 2026).

[17] Vite contributors, “Vite guide: server proxy and build.” [Online]. Available: https://vite.dev/guide/ (Accessed: Oct. 5, 2026).

[18] S. Ramírez, “FastAPI documentation.” [Online]. Available: https://fastapi.tiangolo.com/ (Accessed: Oct. 5, 2026).

[19] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, May 2015.

[20] Tailwind Labs, “Tailwind CSS documentation.” [Online]. Available: https://tailwindcss.com/docs (Accessed: Oct. 5, 2026).

[21] OWASP Foundation, “HTML5 Security Cheat Sheet: Local Storage.” [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html (Accessed: Oct. 5, 2026).